

Fusion Financial — Proposed Security Hardening

Prepared: 2026-05-17 | **Prepared by:** GLV Dev Agent **Client:** Fusion Financial / Tax Titan | **Domain:** fusionfinancialssm.com **Hosting:** SiteGround GrowBig + GitHub Actions FTP deploy | **Stack:** React SPA (Lovable) + Supabase **Client status:** Active (favour engagement — not currently paying)

> **Elevated sensitivity:** Fusion handles financial and tax data for clients. Even though the public-facing site is a React SPA (not a database-driven WP site), the Supabase backend (`ymzrzuknmckblxzjqhhi`) contains bookkeeping records. Layers 1 and 4 carry higher weight here than on Titan. RLS policy audit is non-optional.

Layer 1 — Error Tracking & Logs

Current State

None. No Sentry or equivalent. JS errors in the React SPA are invisible. The Supabase backend has built-in logs (Edge Function logs, database slow queries) but these are not being monitored or alerted. The GitHub Actions FTP deploy pipeline has no error alerting beyond email-on-failure from GitHub.

Proposed Addition

Sentry Free tier for React + manual Supabase log review cadence

React frontend (same pattern as Titan):

```
npm install @sentry/react
```

```
// src/main.tsx
import * as Sentry from "@sentry/react";

Sentry.init({
  dsn: "https://<key>@o<org>.ingest.sentry.io/<project>",
  environment: import.meta.env.MODE,
  // For a financial services site, avoid logging sensitive user input in breadcrumbs:
  beforeBreadcrumb(breadcrumb) {
    if (breadcrumb.category === 'ui.input') return null; // Don't log form keystrokes
    return breadcrumb;
  },
});
```

```
<Sentry.ErrorBoundary fallback={<p>Something went wrong. Please refresh or call us at (703) 754-1474</p>}>
  <App />
</Sentry.ErrorBoundary>
```

Supabase monitoring (free, no additional tooling):

- Enable email notifications in Supabase dashboard: Settings → Notifications → Enable all
- Review Supabase Logs weekly: Logs → API logs (look for 4xx/5xx spikes), Database logs (slow queries), Auth logs (failed login attempts)
- Set up Supabase log drain to send API errors to Sentry (optional, Supabase Pro required — defer until client is paying)

Source maps in CI:

```
# .github/workflows/deploy.yml
- name: Upload Sentry source maps
  env:
    SENTRY_AUTH_TOKEN: ${ secrets.SENTRY_AUTH_TOKEN }
  run: npx @sentry/cli releases files ${ github.sha } upload-sourcemaps ./dist
```

Estimated effort	2.5h (npm install, Sentry init with privacy config, ErrorBoundary, CI source maps, Supabase notification setup, cadence doc)
Risk to live site	None — monitoring layer. Privacy config (<code>beforeBreadcrumb</code>) ensures no sensitive financial form data leaks to Sentry.
Cost	\$0 (Sentry Free, Supabase Free email notifications)

Layer 2 — Rate Limiting

Current State

Cloudflare is already proxying fusionfinancialssm.com (zone confirmed in context). No WAF rate limiting rules configured. The contact/lead form is at full request rate. The Supabase API (ymzrzuknmckblxzjqhhi.supabase.co) is called directly from the frontend — rate limiting at Cloudflare edge won't protect Supabase calls directly, but it limits bot spam on the contact form.

Proposed Addition

Cloudflare WAF rules (already have the zone)

```
# Rule 1: Rate limit contact/lead form submissions
(http.request.method eq "POST" and
  http.request.uri.path contains "/contact" or http.request.uri.path eq "/")
→ Rate Limit: 3 req/min per IP → Block 30min
# Note: More aggressive than Titan because financial services attracts more targeted form

# Rule 2: Block known vulnerability scanners
(http.user_agent contains "sqlmap" or http.user_agent contains "nikto" or
  http.user_agent contains "acunetix" or http.user_agent contains "masscan") → Block

# Rule 3: Managed Challenge on non-CA traffic (stronger than Titan – local client base)
(not ip.geoip.country in {"CA" "US"}) → Managed Challenge
```

Supabase-level rate limiting (separate layer): Supabase Free tier has built-in rate limiting but it's not configurable. Enforce on the application side:

```
// src/lib/supabase.ts – wrap auth calls with client-side rate limit
const SUBMIT_COOLDOWN_MS = 30_000; // 30 seconds between form submits
let lastSubmit = 0;
export function rateLimitedSubmit(fn: () => Promise<void>) {
  const now = Date.now();
  if (now - lastSubmit < SUBMIT_COOLDOWN_MS) throw new Error("Please wait before submitting")
  lastSubmit = now;
  return fn();
}
```

Estimated effort	1h (3 CF WAF rules + client-side rate limit wrapper in Supabase lib)
Risk to live site	Low — POST rate limit at 3/min is tight but appropriate for financial lead form; geo challenge may occasionally affect legitimate US visitors
Cost	\$0

Layer 3 — Uptime Monitoring

Current State

None. No automated monitoring. The site could go down after a failed deploy (FTP issue, Supabase project pause) and remain down until noticed manually.

Special concern: Supabase Free tier projects **pause after 7 days of inactivity**. If the Supabase project pauses, any feature that calls Supabase (likely auth or lead capture) would fail silently from the user's perspective.

Proposed Addition

UptimeRobot Free (batch setup with other clients)

1. Add monitors:

- <https://fusionfinancialssm.com> — HTTP(S), 5-min interval
- <https://ymzrzuknmckblxzjqhhi.supabase.co/rest/v1/> — HTTP(S), 5-min interval (keeps Supabase project awake + alerts on pause)

1. Alert contacts: Email amglvdigital@pm.me + Slack webhook → #internal-dev

2. **Supabase keepalive note:** The UptimeRobot ping to the Supabase REST endpoint serves double duty — it prevents the free-tier project from pausing due to inactivity.

Estimated effort	0.25h (2 monitors, same UptimeRobot account)
Risk to live site	None
Cost	\$0

Layer 4 — Security Hardening

Current State

React SPA = no wp-admin attack surface. However, **Supabase backend contains financial data** — Row Level Security (RLS) policy status is unknown. GitHub Actions uses FTP credentials stored as GitHub Secrets — whether these secrets are scoped correctly is unknown. No formal security review cadence.

Proposed Addition

4a — Supabase RLS audit (highest priority for this client)

This is the most important security item for Fusion. Financial records in Supabase must not be readable by unauthenticated users.

```
-- Run in Supabase SQL Editor for each table:
SELECT schemaname, tablename, rowsecurity
FROM pg_tables
WHERE schemaname = 'public';

-- For each table, verify RLS is ON:
ALTER TABLE <table_name> ENABLE ROW LEVEL SECURITY;

-- Verify no "open" policies exist:
SELECT * FROM pg_policies WHERE schemaname = 'public';

-- Example locked-down policy (read own records only):
CREATE POLICY "Users see own records" ON bookkeeping_entries
FOR SELECT USING (auth.uid() = user_id);
```

Current Supabase tables: 45 tables at project `ymzrzuknmckblxzjqhhi` (ca-central-1). Full RLS audit needed. Flag any public-read tables to Aiden before proceeding.

4b — GitHub Actions secrets audit

```
# Check what secrets are in the repo
gh secret list --repo glvcrypto/fusionfinancial
```

- Verify FTP credentials are scoped to `fusionfinancialssm.com` only
- Add secret scanning: Settings → Security → Secret scanning (catches accidentally committed keys)
- Add branch protection on `main` : require PR review

4c — SiteGround + Cloudflare account 2FA (same as Titan — same account)

- SiteGround 2FA covers all 3 client sites simultaneously
- Cloudflare 2FA covers Titan + Fusion simultaneously

4d — Dependency audit + Supabase SDK version check

```
npm audit
npx npm-check-updates --filter "@supabase/supabase-js" --interactive
```

Supabase client SDK updates sometimes include auth security patches — stay current.

4e — Review frontend API exposure

```
# Check that Supabase anon key is the only key in the frontend
grep -r "service_role" src/ || echo "OK — no service role key in frontend"
grep -r "supabase" src/ --include="*.ts" --include="*.tsx" | grep "key\\|secret\\|token"
```

The `service_role` key must never be in frontend code — it bypasses RLS entirely.

Estimated effort	3h (Supabase RLS audit is the bulk — 45 tables; GitHub secrets audit, dep scan, API key check)
Risk to live site	Medium for RLS changes — enabling RLS on a table that currently serves unauthenticated reads will break that feature immediately. Audit first, fix + test on staging before production.
Cost	\$0

Layer 5 — Rollback Runbook

Current State

GitHub Actions FTP deploy exists (`glvcrypto/fusionfinancial`). No documented rollback procedure. A bad deploy pushes files directly to SiteGround via FTP — the previous version is not automatically kept.

Proposed Rollback Runbook

Fusion Financial — Site Rollback Runbook v1

Prerequisites:

- GitHub access to `glvcrypto/fusionfinancial` (Aiden or Ben)
- SiteGround FTP access (credentials in Site Tools → FTP Accounts)
- Supabase dashboard access: app.supabase.com (project `ymzrzuknmckblxzjqhhi`)
- Cloudflare dashboard access (for cache purge)

Scenario A — Bad frontend deploy (React SPA broken)

```
# 1. Identify last known good commit
git log --oneline main | head -10

# 2. Build the previous version locally
git checkout <last-good-commit-hash>
npm install && npm run build

# 3. Deploy via FTP to SiteGround (manual override of GitHub Actions)
# Use FileZilla or SiteGround File Manager
# Upload dist/* → public_html/ (or the configured web root)

# 4. Purge Cloudflare cache
# Cloudflare dashboard → fusionfinancialssm.com → Caching → Purge Everything

# 5. Verify: curl -I https://fusionfinancialssm.com
```

Scenario B — GitHub Actions deploy pipeline failed

1. Check Actions run log: github.com/glvcrypto/fusionfinancial/actions
2. Common failure: FTP credentials expired (SiteGround rotates FTP passwords on account changes)
3. Fix: SiteGround → Site Tools → FTP Accounts → Change password → Update GitHub Secret `FTP_PASSWORD`
4. Re-trigger: Actions → Re-run failed jobs

Scenario C — Supabase incident (backend down or project paused)

1. Check Supabase status: status.supabase.com
2. If project paused (Free tier inactivity): app.supabase.com → project `ymzrzuknmckblxzjqhhi` → Restore Project
 - Note: Restore takes ~2 minutes; UptimeRobot monitor will alert within 5 min of pause
1. If Supabase is having an incident: Monitor status.supabase.com, no action available from our end
2. Consider upgrading to Supabase Pro (\$25/mo) to eliminate pause risk if Fusion becomes a paying client

Scenario D — Data corruption or accidental record deletion

1. Supabase Free tier includes 7-day Point-in-Time Recovery (PITR) via dashboard
2. app.supabase.com → project → Database → Backups
3. Restore to a point before the incident

4. **CRITICAL:** Before restore, export current data snapshot for forensics:

```
`` sql -- Run in SQL Editor, export result SELECT * FROM <affected_table> ORDER BY
created_at DESC LIMIT 500; ``
```

Emergency contacts:

- SiteGround: my.siteground.com/chat (24/7)
- Supabase: supabase.com/support (free tier: community forum; paid: ticketed support)
- Aiden: amglvdigital@pm.me / Telegram

Estimated effort	1h (write + test Scenario A manually; document Supabase restore path)
Risk to live site	None — documentation only
Cost	\$0

Summary

Layer	Current State	Proposed	Effort	Risk	Cost
1. Error Tracking	None	Sentry React SDK (privacy-safe config) + Supabase notifications	2.5h	None	\$0
2. Rate Limiting	Cloudflare zone active, no rules	WAF rules (stricter than Titan) + client-side Supabase rate limit	1h	Low	\$0
3. Uptime Monitoring	None	UptimeRobot (2 monitors: site + Supabase keepalive)	0.25h	None	\$0
4. Security Hardening	Unknown	Supabase RLS audit (45 tables) + GH secrets audit + 2FA + dep scan	3h	Med (RLS changes)	\$0
5. Rollback Runbook	None	4-scenario runbook (frontend/pipeline/Supabase/data)	1h	None	\$0
Total			~7.75h		\$0

Priority order for Fusion specifically: Layer 4 (RLS audit) > Layer 3 (Supabase keepalive monitor) > Layer 2 > Layer 1 > Layer 5. Financial data exposure risk makes the RLS audit the single most important item on this list.

All items default build hardening — not billable.